LINQ (Language Integrated Query)



Soyeb Ghachi

Technical Trainer | Mentor |SME

Overview



LINQ (Language Integrated Query)

Advantages of LINQ

LINQ API

Enumerable

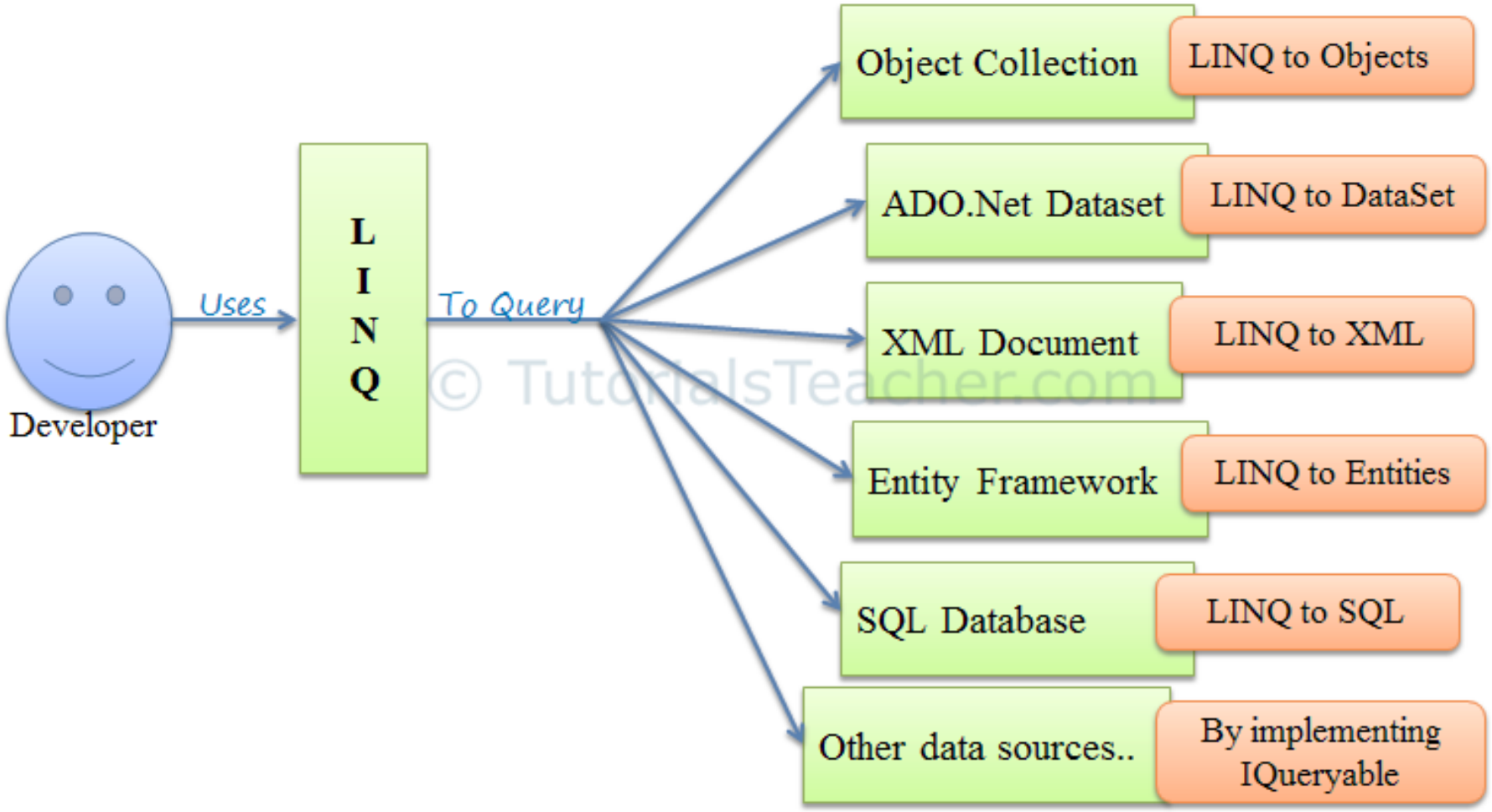
Queryable

LINQ Lambda Expressions

LINQ Query Operators

LINQ (Language Integrated Query)

- LINQ (Language Integrated Query) is uniform query syntax in C# and VB.NET to retrieve data from different sources and formats.
- It is integrated in C# or VB, thereby eliminating the mismatch between programming languages and databases, as well as providing a single querying interface for different types of data sources.
- For example, SQL is a Structured Query Language used to save and retrieve data from a database. In the same way, LINQ is a structured query syntax built in C# and VB.NET to retrieve data from different types of data sources such as collections, ADO.Net DataSet, XML Docs, web service and MS SQL Server and other databases.


Developer
Uses
L
I
N
Q
To Query
Object Collection
LINQ to Objects
ADO.Net Dataset
LINQ to DataSet
XML Document
LINQ to XML
Entity Framework
LINQ to Entities
SQL Database
LINQ to SQL
Other data sources..
By implementing IQueryable


Objects
Returns
{ Linq Query }
Execute Query
Retrieve Result
Data Source

Advantages of LINQ

Advantages of LINQ

- **Familiar language:** Developers don’t have to learn a new query language for each type of data source or data format.
- **Less coding:** It reduces the amount of code to be written as compared with a more traditional approach.
- **Readable code:** LINQ makes the code more readable so other developers can easily understand and maintain it.
- **Standardized way of querying multiple data sources:** The same LINQ syntax can be used to query multiple data sources.
- **Compile time safety of queries:** It provides type checking of objects at compile time.
- **IntelliSense Support:** LINQ provides IntelliSense for generic collections.
- **Shaping data:** You can retrieve data in different shapes.

LINQ Lambda Expressions

In LINQ, **Lambda expression** is a function without a name. It makes the syntax short and clear. Though it is not as readable as a LINQ query it is equally important as the LINQ query and gets convert to lambda internally. Its scope is limited to where it is used as an expression. We cannot reuse it afterward.

```
List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
List<int> evenNumbers = numbers.Where(x => x % 2 == 0).ToList();
Console.WriteLine(string.Join(", ", evenNumbers));  // Output: 2, 4, 6, 8, 10
```

- **Query using Linq query and Linq Operator (extension methods).**

```
using System;
using System.Linq;
public class LINQExample
{
    public static void Main(string[] args)
    {
        // Sample data
        int[] numbers = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

        // LINQ query using LINQ syntax
        var querySyntaxResult = from number in numbers where number % 2 == 0 select number;
        Console.WriteLine("Results using LINQ syntax:");
        foreach (var number in querySyntaxResult)
        {
            Console.WriteLine(number);
        }
       // LINQ query using LINQ operators/extension methods
        var methodSyntaxResult = numbers.Where(number => number % 2 == 0);
        Console.WriteLine("Results using LINQ operators:");
        foreach (var number in methodSyntaxResult)
        {
            Console.WriteLine(number);
        }
    }}
```

Enumerable

LINQ API

We can write LINQ queries for the classes that implement IEnumerable<T> or IQueryable<T> interface.

The *System.Linq* namespace includes the following classes and interfaces require for LINQ queries.

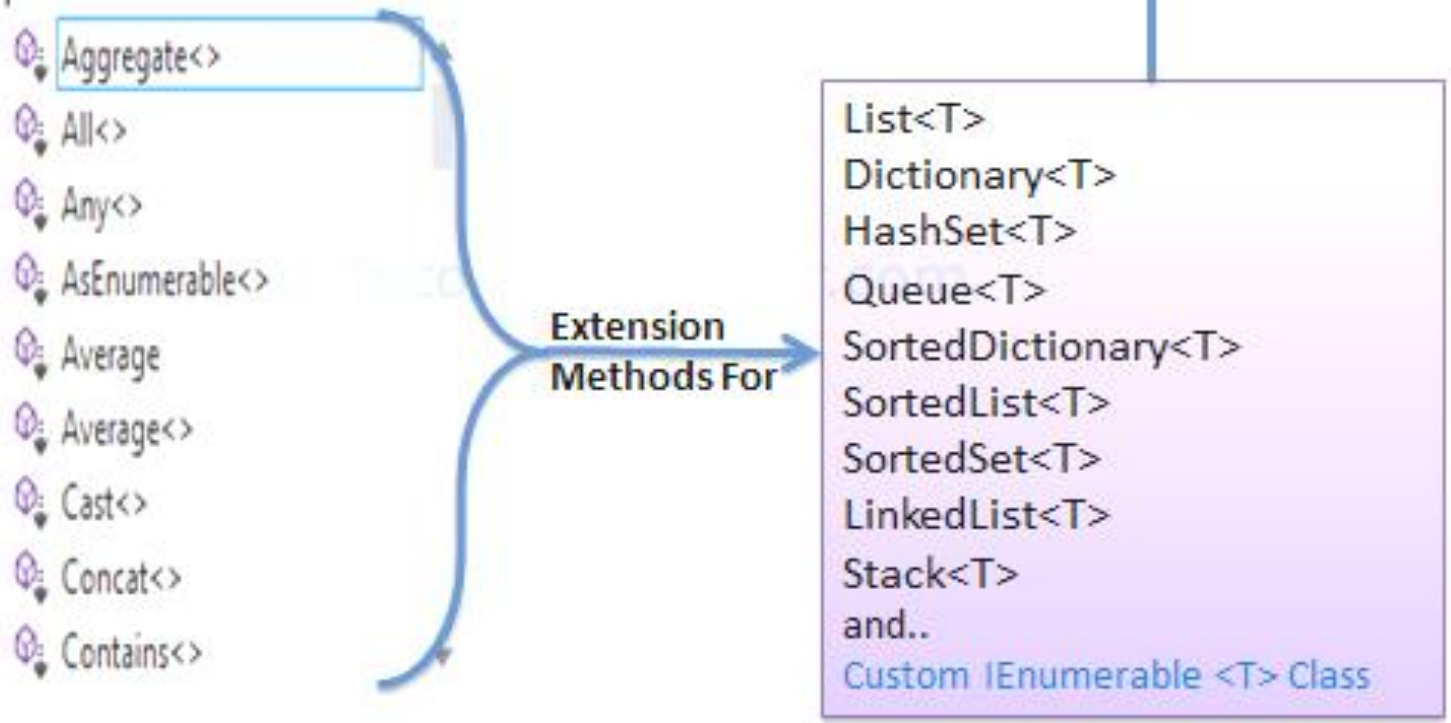
https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/linq/

Enumerable

The Enumerable class includes extension methods for the classes that implement IEnumerable<T> interface, for example all the built-in collection classes implement IEnumerable<T> interface and so we can write LINQ queries to retrieve data from the built-in collections.

https://learn.microsoft.com/en-us/dotnet/api/system.linq.enumerable?view=net-7.0

```
System.Linq.Enumerable.
```



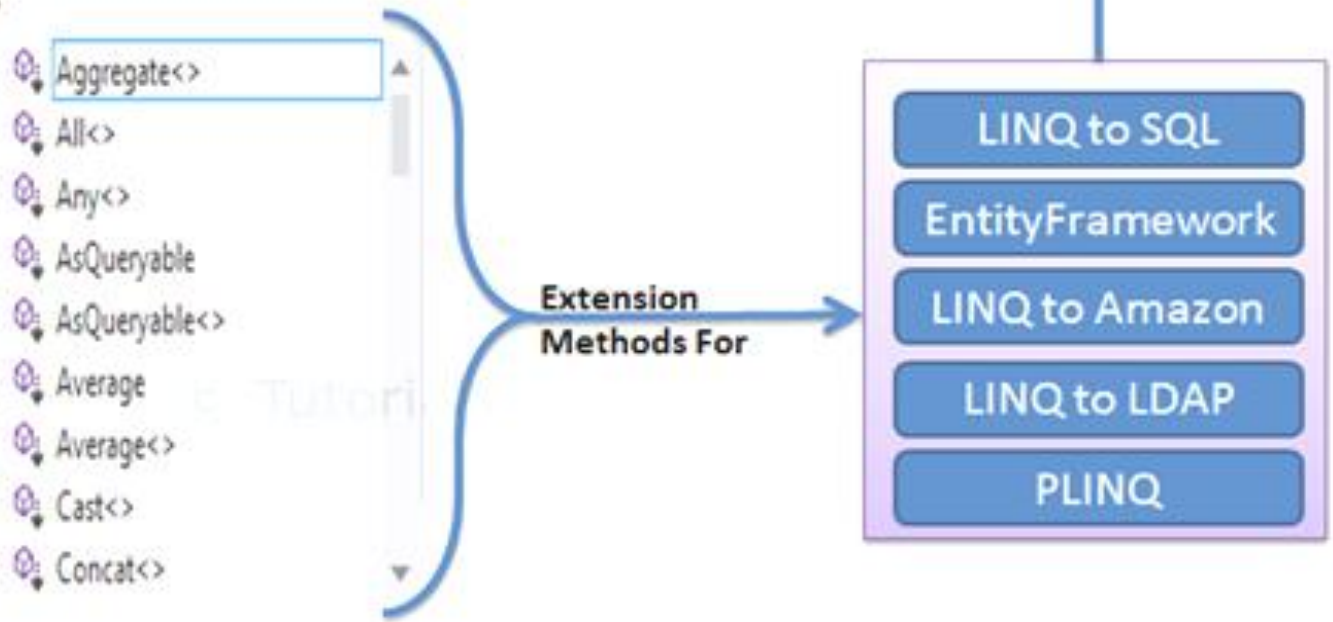
Queryable

Queryable

The Queryable class includes extension methods for classes that implement IQueryable<t> interface. The IQueryable<T> interface is used to provide querying capabilities against a specific data source where the type of the data is known. For example, Entity Framework api implements IQueryable<T> interface to support LINQ queries with underlaying databases such as MS SQL Server.

https://learn.microsoft.com/en-us/dotnet/api/system.linq.iqueryable?view=net-7.0

System.Linq.Queryable.



Standard Query Operators

Classification	Standard Query Operators
Filtering	Where, OfType
Sorting	OrderBy, OrderByDescending, ThenBy, ThenByDescending, Reverse
Grouping	GroupBy, ToLookup
Join	GroupJoin, Join
Projection	Select, SelectMany
Aggregation	Aggregate, Average, Count, LongCount, Max, Min, Sum
Quantifiers	All, Any, Contains
Elements	ElementAt, ElementAtOrDefault, First, FirstOrDefault, Last, LastOrDefault, Single, SingleOrDefault
Set	Distinct, Except, Intersect, Union
Partitioning	Skip, SkipWhile, Take, TakeWhile
Concatenation	Concat
Equality	SequenceEqual
Generation	DefaultEmpty, Empty, Range, Repeat
Conversion	AsEnumerable, AsQueryable, Cast, ToArray, ToDictionary, ToList

Example: Standard Query Operators

```
class Student
    {
        public int Age { get; set; }
        public int StudentID { get; set; }
        public string StudentName { get; set; }
    }
```

```
class Program
   {
      static void Main(string[] args)
      {
         IList<Student> studentList = new List<Student>() {
      new Student() { StudentID = 1, StudentName = "John", Age = 13} ,
      new Student() { StudentID = 2, StudentName = "Moin",  Age = 21 } ,
      new Student() { StudentID = 3, StudentName = "Bill",  Age = 18 } ,
      new Student() { StudentID = 4, StudentName = "Ram" , Age = 20} ,
      new Student() { StudentID = 5, StudentName = "Ron" , Age = 15 }
   };
         var student = from s in studentList where s.Age > 20 select s;
         foreach (Student std in student)
         {
            Console.WriteLine(std.StudentName);
         }
        var student1 = studentList.Where(s => s.Age > 20).ToList<Student>();
         foreach (Student std in student1)
         {
            Console.WriteLine(std.StudentName);
         }
         Console.ReadLine();
      }
   }
```

Sorting Operators: OrderBy & OrderByDescending: OrderBy

```
class Program
   {
      static void Main(string[] args)
      {
         IList<Student> studentList = new List<Student>() {
   new Student() { StudentID = 1, StudentName = "John", Age = 18 } ,
   new Student() { StudentID = 2, StudentName = "Steve",  Age = 15 } ,
   new Student() { StudentID = 3, StudentName = "Bill",  Age = 25 } ,
   new Student() { StudentID = 4, StudentName = "Ram" , Age = 20 } ,
   new Student() { StudentID = 5, StudentName = "Ron" , Age = 19 }
};

var orderByResult = from s in studentList orderby s.StudentName select s;
var orderByDescendingResult = from s in studentList
                              orderby s.StudentName descending
                              select s;
         Console.ReadLine();
      }
   }
```

Grouping Operators: GroupBy

```
class Program
   {
      static void Main(string[] args)
      {
         IList<Student> studentList = new List<Student>() {
      new Student() { StudentID = 1, StudentName = "John", Age = 18 } ,
      new Student() { StudentID = 2, StudentName = "Steve",  Age = 21 } ,
      new Student() { StudentID = 3, StudentName = "Bill",  Age = 18 } ,
      new Student() { StudentID = 4, StudentName = "Ram" , Age = 20 } ,
      new Student() { StudentID = 5, StudentName = "Abram" , Age = 21 }
   };
         var groupedResult = from s in studentList group s by s.Age;

         //iterate each group
         foreach (var ageGroup in groupedResult)
         {
       Console.WriteLine("Age Group: {0}", ageGroup.Key); //Each group has a key

            foreach (Student s in ageGroup) // Each group has inner collection
               Console.WriteLine("Student Name: {0}", s.StudentName);
         }} }
```

Projection Operator: Select

```
class Program
   {
      static void Main(string[] args)
      {
         IList<Student> studentList = new List<Student>() {
   new Student() { StudentID = 1, StudentName = "John" },
   new Student() { StudentID = 2, StudentName = "Moin" },
   new Student() { StudentID = 3, StudentName = "Bill" },
   new Student() { StudentID = 4, StudentName = "Ram" },
   new Student() { StudentID = 5, StudentName = "Ron" }
};

         var selectResult = from s in studentList
                            select s.StudentName;
         Console.ReadLine();
      }
```

Quantifier Operators: All, Any

```
class Program
   {
      static void Main(string[] args)
      {
         IList<Student> studentList = new List<Student>() {
      new Student() { StudentID = 1, StudentName = "John", Age = 18 } ,
      new Student() { StudentID = 2, StudentName = "Steve",  Age = 15 } ,
      new Student() { StudentID = 3, StudentName = "Bill",  Age = 25 } ,
      new Student() { StudentID = 4, StudentName = "Ram" , Age = 20 } ,
      new Student() { StudentID = 5, StudentName = "Ron" , Age = 19 }
   };

         bool areAllStudentsTeenAger = studentList.All(s => s.Age > 12 && s.Age < 20);
         bool isAnyStudentTeenAger = studentList.Any(s => s.Age > 12 && s.Age < 20);
         Console.WriteLine(areAllStudentsTeenAger);
         Console.WriteLine(isAnyStudentTeenAger);
         Console.ReadLine();
      }
   }
```

Aggregation Operators: Avg,Count,Max,Sum

```
class Program
   {
      static void Main(string[] args)
      {
         IList<int> intList = new List<int> () { 10, 20, 30 };

         var avg = intList.Average();

         Console.WriteLine("Average: {0}", avg);
         Console.ReadLine();
      }
   }
```

- **Average**

```
class Program
   {
      static void Main(string[] args)
      {
         IList<Student> studentList = new List<Student>() {
            new Student() { StudentID = 1, StudentName = "John", Age = 13 } ,
      new Student() { StudentID = 2, StudentName = "Moin", Age = 21 } ,
      new Student() { StudentID = 3, StudentName = "Bill", Age = 18 } ,
      new Student() { StudentID = 4, StudentName = "Ram", Age = 20 } ,
      new Student() { StudentID = 5, StudentName = "Ron", Age = 15 }
     };

         var avgAge = studentList.Average(s => s.Age);

         Console.WriteLine("Average Age of Student: {0}", avgAge);
         Console.ReadLine();
      }
   }
```

- **Count**

```
class Program
   {
      static void Main(string[] args)
      {
         IList<Student> studentList = new List<Student> () {
            new Student() { StudentID = 1, StudentName = "John", Age = 13 } ,
      new Student() { StudentID = 2, StudentName = "Moin", Age = 21 } ,
      new Student() { StudentID = 3, StudentName = "Bill", Age = 18 } ,
      new Student() { StudentID = 4, StudentName = "Ram", Age = 20 } ,
      new Student() { StudentID = 5, StudentName = "Mathew", Age = 15 }
    };

         var totalStudents = studentList.Count();

         Console.WriteLine("Total Students: {0}", totalStudents);

         var adultStudents = studentList.Count(s => s.Age >= 18);

         Console.WriteLine("Number of Adult Students: {0}", adultStudents);
         Console.ReadLine();
      }
   }
```

- **Max**

```
class Program
   {
      static void Main(string[] args)
      {
         IList<Student> studentList = new List<Student> () {
            new Student() { StudentID = 1, StudentName = "John", Age = 13 } ,
      new Student() { StudentID = 2, StudentName = "Moin", Age = 21 } ,
      new Student() { StudentID = 3, StudentName = "Bill", Age = 18 } ,
      new Student() { StudentID = 4, StudentName = "Ram", Age = 20 } ,
      new Student() { StudentID = 5, StudentName = "Ron", Age = 15 }
    };

         var oldest = studentList.Max(s => s.Age);

         Console.WriteLine("Oldest Student Age: {0}", oldest);
         Console.ReadLine();
      }
   }
```

IEnumerable and IQueryable

1. IEnumerable – In-memory data

IEnumerable is commonly used when the data is already loaded into memory, such as a List<T>.

```
using System;
using System.Collections.Generic;
using System.Linq;

class Program
{
   static void Main()
   {
      List<int> numbers = new List<int> { 10, 20, 30, 40, 50 };

      IEnumerable<int> result = numbers.Where(n => n > 25);

      foreach (int number in result)
      {
         Console.WriteLine(number);
      }
   }
}
```

2. IQueryable – Database dataIQueryable is commonly used with databases through Entity Framework Core. The query can be translated into SQL and executed by the database.

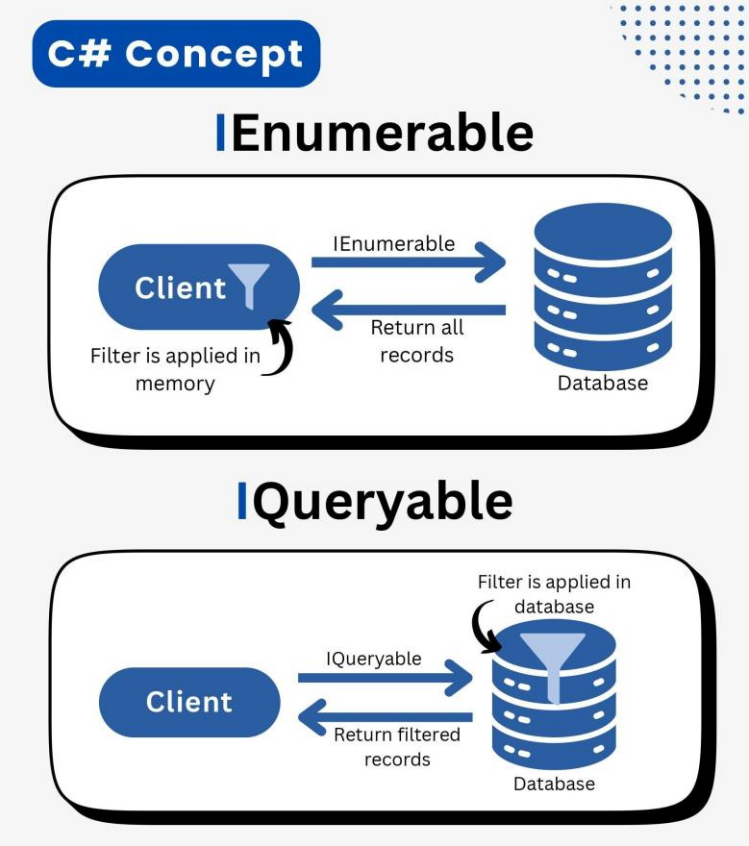
```
IQueryable<Employee> employees = dbContext.Employees;

var result = employees.Where(e => e.Salary > 50000).OrderBy(e => e.Name);
```

The database provider can translate this into SQL similar to:

```
SELECT *
FROM Employees
WHERE Salary > 50000
ORDER BY Name;
```

IEnumerable	IQueryable
Works mainly with in-memory data	Works mainly with data sources such as databases
Filtering happens in application memory	Filtering can happen in the database
Uses System.Collections.Generic	Uses System.Linq
Common with List<T>	Common with Entity Framework
Suitable for already-loaded data	Suitable for querying large external data sources



References

https://learn.microsoft.com/en-us/dotnet/csharp/linq/

https://www.tutorialsteacher.com/linq/what-is-linq

https://www.tutorialsteacher.com/linq/what-is-linq

https://www.c-sharpcorner.com/UploadFile/72d20e/concept-of-linq-with-C-Sharp/

https://www.javatpoint.com/what-is-linq

https://zetcode.com/csharp/linq/

https://www.geeksforgeeks.org/linq-language-integrated-query/